

ERD DESIGN TASKS — COMBINED BRIEF

Database Design Practice Set • Ordered from Easy to Advanced

Level 1 — College Management System

Difficulty Level: **EASY**

Source: Task 2 (Design ERDs)

A college database system is needed to manage students, faculty, courses, hostels, and examinations.

FACULTY

- Identified by F_id.
- Attributes: Name, Mobile_no, Department, Salary.
- Teaches multiple students and handles subjects.

STUDENT

- Identified by S_id.
- Attributes: F_name, L_name, Name, Phone_no, DOB, and Age (derived).
- Can enroll in multiple courses.
- Can belong to a department.
- Can take multiple exams.
- Can live in a hostel.

HOSTEL

- Identified by Hostel_id.
- Attributes: Hostel_name, City, State, Address, Pin_code, No_of_seats.

COURSE

- Identified by Course_id.
- Attributes: Course-name, Duration.
- Courses are handled by departments and have multiple enrolled students.

SUBJECT

- Each subject has a Subject_id and Subject_name.
- Students take subjects which are taught by faculty.

EXAMS

- Identified by Exam_code.
- Attributes: Date, Time, Room.
- Conducted by a department.

DEPARTMENT

- Identified by Department_id.
- Has a D_name.
- Handles courses and conducts exams.

Why this is the easy tier

- Every entity has an obvious single-attribute primary key already given (F_id, S_id, Hostel_id, Course_id, Subject_id, Exam_code, Department_id).
- Relationships are mostly simple 1–M (Department–Course, Department–Exam, Hostel–Student).
- Only one derived attribute (Student Age) and no tricky ternary or self-referencing relationships.

- Good first exercise for practicing entity/attribute/relationship identification before tackling ambiguity.

Level 2 — Airline Information System

Difficulty Level: **INTERMEDIATE**

Source: Task 2 (Design ERDs)

An airline system is needed to manage flight operations, schedules, reservations, and related data.

AIRPORT

- Each airport has a unique `airport_code`.
- It stores city, state, and name.
- An airport can serve as a departure or arrival point for many flight legs.
- An airport may allow landing of different airplane types.

FLIGHT LEG

- Each flight leg is uniquely identified by a `leg_no`.
- A leg has a `scheduled_dep_time` and `scheduled_arr_time`.
- A leg departs from and arrives at an airport.

LEG INSTANCE

- A leg instance is a specific occurrence of a flight leg on a given date.
- It includes: Arrival time, Departure time, Number of available seats.
- It is assigned an airplane and connected to seat reservations.

AIRPLANE TYPE

- Defined by a `type_name`, `company`, and `max_seats`.
- Multiple airplanes can share a type.

AIRPLANE

- Identified by `airplane_id`.
- Has a total number of seats.
- Each airplane belongs to a specific type.
- Assigned to leg instances.

SEAT / RESERVATION

- A reservation is made by a customer with name and phone.
- It is linked to a specific seat number in a leg instance.

FLIGHT & FARE

- A flight includes multiple legs.
- Flights have attributes like Airline, Weekdays, Restrictions.
- Each flight can have multiple fares, with Code and Amount.

Why this is the intermediate tier

- Introduces a weak entity chain: Leg Instance cannot exist without Flight Leg, and Reservation cannot exist without Leg Instance — this identifying-dependency reasoning is a step up from Level 1.
- Requires a 3-way relationship: a Reservation ties together a Customer, a Seat, and a Leg Instance at once.
- Flight–Leg and Flight–Fare are both 1–M but layered on top of the weak-entity chain, so the overall structure is deeper (Flight → Leg → Leg Instance → Reservation).
- Customer is implied but never given a `customer_id`, so you must decide whether Customer becomes its own entity — a modeling judgment call, not a given fact.

Level 3 — Hospital Management System (Main Task)

Difficulty Level: **ADVANCED**

Source: Task 1 (Full Task – Real Company Simulation)

This is the full assignment from the client brief: analyze, classify, design, and document a Hospital Management System covering patients, doctors, departments, appointments, services, medical records, and billing.

System Scope

Patient

- Stores personal and contact information; includes DOB, blood group, gender.
- System should store patient age (derived).
- A patient can have multiple appointments, records, and bills.

Doctor

- Stores specialization and professional details.
- Works in a department; handles appointments; creates medical records; may act as department head.

Department

- Contains multiple doctors; has a head doctor; offers services.
- Undefined: Is head mandatory? Can a doctor manage multiple departments?

Appointment

- Connects patient and doctor; stores date, time, status, type.
- May include services; may result in a medical record.
- Undefined: Can it exist without services?

Service

- Has name, type, price; used during appointments.
- Undefined: Does price change over time?

Medical Record

- Stores diagnosis and treatment; linked to patient and doctor; related to an appointment.

Billing

- Stores payment details; linked to appointment; calculates total based on services.
- Undefined: Can payment be partial?

Critical Business Logic

- An appointment can include multiple services, and a service can appear in multiple appointments (M:M).
- The system must track how many times a service is used.
- You must decide how to design this — including where the Quantity attribute belongs.

Main Task Requirements

1. System Analysis — identify Entities, Attributes, Relationships.
- Required Thinking (mandatory, before any drawing): classify every component as Entity / Attribute / Relationship / Relationship Attribute / Derived / Multi-valued / Strong or Weak, in a table of Item | Classification | Reason.
2. Logical Design — Primary Keys, Candidate Keys, relationship types, participation; resolve the Appointment ↔ Service case explicitly (identify the relationship, design it correctly, decide where Quantity belongs).
3. ERD Design — full ERD with entities, attributes, keys, relationships, and cardinality.
4. System Behavior Analysis — explain how the design handles appointment cancellation, a doctor leaving, partial payments, and service price changes.

- 5. Design Improvements — propose new entities or a better structure.

Required Submission Structure

Deliverables must be organized exactly as follows (an incomplete structure is treated as an incomplete submission):

- Hospital-System-ERD/ ← main project folder
 - 01-Analysis/ — identified entities, attributes, relationships
 - 02-Logical-Design/ — keys and relationships
 - 03-ERD/ — final ERD diagram (PNG/JPG or PDF)
 - 04-Scenarios/ — real-world behavior analysis
 - 05-Required-Thinking/ — the classification task (most important part)
 - README.md — project overview, key design decisions, challenges faced

Why this is the advanced tier

- It is not just an ERD drawing — it requires a full mandatory classification pass (Entity/Attribute/Relationship/Derived/Multi-valued/Strong-Weak) before any diagram is allowed.
- Several business rules are deliberately left undefined (head mandatory?, doctor in multiple departments?, price changes?, partial payments?) — you must make and justify design decisions, not just transcribe requirements.
- Contains a genuine M:M relationship (Appointment ↔ Service) with an attribute-placement decision (where Quantity/usage-count lives), which is one of the classic hard cases in relational modeling.
- Demands post-design reasoning: cancellation, doctor turnover, partial payments, and price history are dynamic scenarios, not static structure.
- Requires professional documentation discipline: a fixed folder structure and a README summarizing decisions and challenges, mirroring how a real engineering team delivers design work.

Suggested order of attack: College Management System → Airline Information System → Hospital Management System (main task).